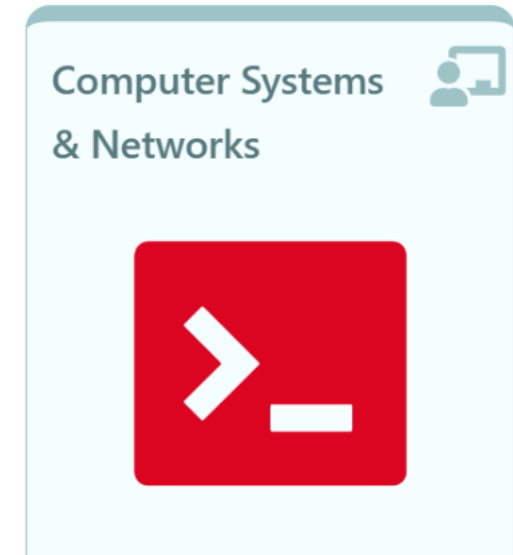


Computer Systems & Networks



Week 3

00: Module introduction



Linux · Computer Networks

01: Intro and Command Line...



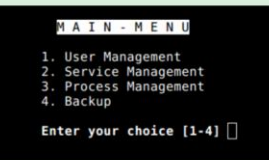
Scripting · Permissions · Navigation

02: File and Text Manipulation



Redirection · grep · Positional Param · Exit Codes

03: Loops, Menus, sed



case · select · loops · sed · awk · 5% HW

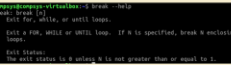
03: if, while, for, break & continue; case & select for menus; using sed editor

01: Conditionals and Loops



test · if · for · while · until

02: Loop manipulation



break · continue · case · select

03: sed



Stream editor · find and replace

Lab



Loops · Menus · sed

Lab

Computer Systems & Networks

Current Week

2

Search

Computer Syste...

03: Loops, Men...

Week 3 Class 1...

Lab

Advancing Scripting Skills

01. Conditionals
02. Loops
03. Menu driven scripts
04. sed
05. EXER Add new records
06. Cisco NetAcad

SOLUTIONS FROM WEEK 2:

Arithmetic Operators

- Create a **numEmp** file that returns basic details from **employee.txt**

```
empCount=$(cat ~/employee.txt | wc -l)
echo "Today on `date` there are ${empCount} employees."
```

Some examples for reference i.e. to multiply is included here

Addition

Computes addition of two numbers provided as operands.

```
#!/bin/bash
x=$(( 15 + 8 ))
echo $x
```

Week 2 LAB Recap

REMEMBER: Use Resources Freely Available to you

Share resources via Slack

- Here's *one of many* examples

This is a link but you'll have
to download the class
contents

- *links are not accessible directly
from tutors*

Ryans Tutorials

Bash Scripting Tutorial - 4. Arithmetic of 8

Arithmetic!

It all adds up.



Arithmetic Expansion **`$((...))`** v's older **`expr`**

- use **`$((...))`** to do math calculations

```
~$ echo "$(( 3 + (4+5) ))"
```

Why use **`$((... ))`**

- ✓ Inside **`((...))`**, you don't need to use `$` to reference variables.
- ✓ Supports all integer arithmetic operators, as per next slide.
- ✓ more forgiving about spaces.
- ✓ **`((...))`** is a **built-in Bash feature**, so it's *faster* than external commands like **`expr`**

- What would be a more efficient way of writing the following?

```
compsys@compsys-virtualbox:~$ myvar=100  
compsys@compsys-virtualbox:~$ expr ${myvar}  
300
```

```
$myvar=100;  
$echo $((myvar * 3))
```

Arithmetic Expansion **`$((...))`** v's older **`expr`**

```
#!/bin/bash
```

```
x=10
```

```
y=3
```

```
echo $(( x + y )) # addition
```

```
echo $(( $x + $y )) # also valid
```

```
echo $(( x - y )) # subtraction
```

```
echo $(( $x - $y )) # also valid
```

```
echo $(( x * y )) # multiplication
```

```
echo $(( $x * $y )) # also valid
```

```
echo $(( x / y )) # division
```

```
echo $(( $x / $y )) # also valid
```

```
echo $(( x ** y )) # exponentiation
```

```
echo $(( $x ** $y )) # also valid
```

```
echo $(( x % y )) # modular division
```

```
echo $(( $x % $y )) # also valid
```

```
(( x += 4 )) # increment variable by constant
```

```
echo $x
```

```
(( x -= 4 )) # decrement variable by constant
```

```
echo $x
```

```
(( x *= 4 )) # multiply variable by constant
```

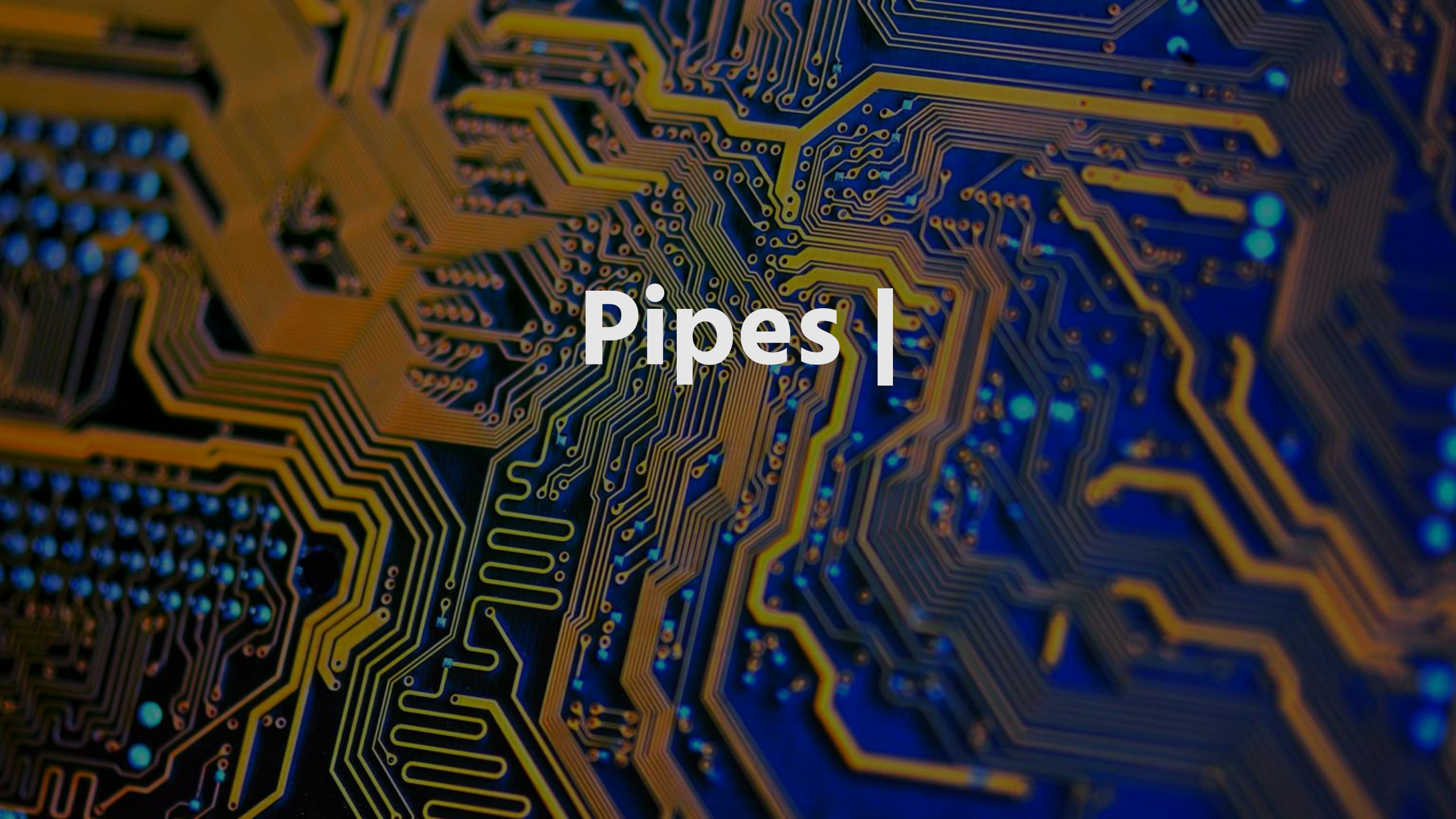
```
echo $x
```

```
(( x /= 4 )) # divide variable by constant
```

```
echo $x
```

```
(( x %= 4 )) # Remainder of Dividing Variable by Constant
```

```
echo $x
```


The background of the image is a detailed, close-up view of a printed circuit board (PCB). The board is dark blue or black, with a complex network of fine, gold-colored conductive traces. These traces form various patterns, including straight lines, curves, and dense clusters. Interspersed among the traces are numerous small, glowing blue circular components, likely surface-mount LEDs or capacitors, which add a vibrant, futuristic feel to the image. The lighting is dramatic, highlighting the metallic sheen of the gold traces and the bright glow of the blue components.

Pipes I

Working with Pipelines



- One of the most powerful Linux commands
 - Syntax: `|`
 - takes **output** from one command and **uses it as input** for another
- ✓ Can link/chain multiple pipes

Main purpose: filtering

- The most popular use for pipes involves the commands **sort** and **grep**

DON'T CONFUSE pipe (|) redirection with file redirection (>) and (<).

cmd1 > file

or

cmd1 < file

→ File redirection either sends output to a file

→ instead of STDOUT (screen) **OR**

→ pulls input from file instead of STDIN

→ (keyboard)

pipe and sort

sort file on the basis of 1st three fields:

```
$ sort -t"," -k1,3 employee.txt
```

Where

- t** = option is used to provide the delimiter, assuming it's a comma
- k** = specify the keys on the basis of which the sorting has to be done.
format of '-k' is : '-km,n' (*m* = the starting key and *n* = the ending key)
- f** would sort and ignore case
- n** would sort numerically
- c** would sort and remove duplicates

grep pipe and sort

We want to return only the lines that *do not* contain the characters 'Manager', but the result should be in *reverse* order.

```
$ cat employee.txt | grep -vi Manager | sort - r
```

Counting files

- For example, you can list the number of files in the current directory:

\$ls

#send output to STDOUT (screen) by default

\$ls -l | less

pipe to less to make output easier to read/scrollable

\$ ls | head -3 | tail -1

#stack(/chain) commands using pipes

\$ls | head -3 | tail -1 > myoutput

#combine with redirection

Count lines

\$ wc -l employee.txt #How many lines are in *employee.txt*?

How to pipe the output of **ls** command into **wc**:

(i) **\$ls ~ | wc -l** #how many files are home

(ii) Pipe this to `less` to inspect, q to quit

(iii) \$ _____ #how many lines are in the /etc

Pipe Syntax

|

vertical line

View history of
commands, in
scrollable
format:

\$history | less

Try out

\$history | head -5

Count the number
of lines in the
home directory

\$ls ~ | wc -l

Week 3

space style 1 space

```
if [ condition1 ]  
then  
    statements  
elif [ condition2 ]  
then  
    statements  
else  
    statements  
fi
```

style 2

```
if [ condition1 ]; then  
    statements  
elif [ condition2 ]; then  
    statements  
else  
    statements  
fi
```

Conditional Statements

test



test command – is true/false

```
Caroline@ICTSkills>~$ test 10 -gt 55; echo $?  
1
```

Tests can be combined with logical **AND** and **OR**

```
compsys@compsys-virtualbox:~$ test 56 -gt 55 && echo true || echo false  
true  
compsys@compsys-virtualbox:~$ test 6 -gt 55 && echo true || echo false  
false
```

Re-written using square brackets

```
compsys@compsys-virtualbox:~$ [ 56 -gt 55 ] && echo true || echo false
```


Arithmetic Operators

+ -	Addition, subtraction
++ --	Increment, decrement
* / %	Multiplication, division, remainder
**	Exponentiation

-eq	→	_____
-ge	→	_____
-gt	→	_____
-le	→	_____
-lt	→	_____
-ne	→	_____

If BASH double parenthesis i.e. **(())** are not used
then the **test** command must be used to **compare** integer
variables

Try out this weeks Lab

01: Conditionals and Loops

LOOPS REPEAT ACTIONS... SO YOU DON'T HAVE TO

test · if · for · while · until

02: Loop manipulation

break · continue · case · select

03: sed

Stream editor · find and replace

Lab

#!/bin/bash

Loops · Menus · sed

Advancing Scripting Skills

1. Conditionals

2. Loops

3. Menu driven scripts

4. sed

5. EXER Add new records

6. Cisco NetAcad

03: if, while, for, break & continue; case & select for menus; using sed editor

Combining Multiple Operators

It can be convenient at the beginning of execution to **check if a particular file exists** and create if not.

Here's the steps:

1. first check if a file **sample.txt** exists in your home directory (try make use of [**-f filename**] instead of the [**-d filename**] used above)
2. if it exists, print a message to STDOUT (**echo "File exists."**).
3. If not, create the file (**touch ~/sample.txt**)

```
compsys@compsys-virtualbox:~$ ls -l samp*
ls: cannot access 'samp*': No such file or directory
compsys@compsys-virtualbox:~$ [ -f ~/sample.txt ] && echo "File exists." || touch ~/sample.txt
compsys@compsys-virtualbox:~$ ls -l samp*
-rw-rw-r-- 1 compsys compsys 0 Sep  8 11:55 sample.txt
compsys@compsys-virtualbox:~$
```

test statement

Often used to testing verifying conditions, including:

if



if statement

= does a string comparison and

-eq does a numerical comparison

if [*test*]

then

commands

fi

```
if [ "$userinput" != "$password" ];  
then  
echo "Oops; Wrong Password Entered!"  
exit 1  
fi
```

How to use exit codes in scripts with an *if*

- Return a message to the user if this file called “employee.txt” ran successfully

What will be the output?

```
#!/bin/bash

cat employee.txt

if [ $? -eq 0 ]
then
    echo "This file $0 ran OK"
    exit 0
else
    echo "File failed"
    exit 1
fi
```

if/then/else

- Sometimes we may have a series of conditions that may lead to different paths.

```
if [ test ]  
then commands  
else commands  
fi
```

```
#!/bin/bash  
if [ -f employee.bak ]  
    then echo employee.bak regular  
file exists!  
    else echo employee.bak not found!  
fi
```

How to use **exit codes** in scripts with an *if/then/else*

\$? = the **exit status** of the last command executed

```
if [ $? -eq 0 ]; then
```

```
    # do something as the last command executed OK
```

```
else
```

```
    # do something else as last command executed with error
```

```
fi
```

```
exit
```

if then elif

- You can nest a new **if** inside an **else** with **elif**

```
#!/bin/bash
count=42
if [ $count -eq 42 ]
then
    echo "42 is correct."
elif [ $count -gt 42 ]
then
    echo "Too much."
else
    echo "More!"
fi
```


\$# Shell Variable

- is the number of arguments taken in

\$echo \$# #outputs the number of positional parameters of your script.

```
if [ $# -eq 1 ]
then
    echo $1
else
    echo "invalid argument please pass only one argument, (Y or N): "
fi
exit
```

Try out this weeks Lab

03: if, while, for, break & continue; case & select for menus; using sed editor

01: Conditionals and Loops



LOOPS REPEAT ACTIONS...
SO YOU DON'T HAVE TO

test · if · for · while · until

02: Loop manipulation

```
break, break {n}
exit, exit {n}
exit {n}
exit {n}
exit {n}
exit {n}
exit {n}
exit {n}
exit {n}
exit {n}
```

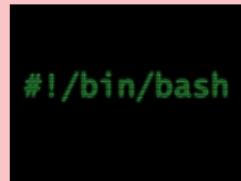
break · continue · case · select

03: sed



Stream editor · find and replace

Lab



Loops · Menus · sed

Advancing Scripting Skills

01. Conditionals

02. Loops

03. Menu driven scripts

04. sed

05. EXER Add new records

06. Cisco NetAcad

if statement

1. Make a backup of **employee.txt** called **employee.bak**

```
cp employee.txt employee.bak
```

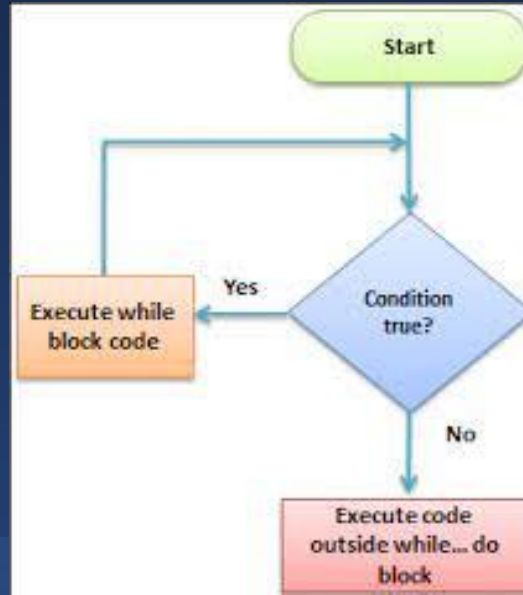
2. Write a script **rmOld** which checks if an old temporary file exists called **employee.bak** and if so, then delete this

HINT: **-a filename** or **-e filename** will check if **filename** exists.

```
if [ -a employee.bak ]; then
rm -f employee.bak
fi
```

3. If **./bashrc** is a regular file (use the **-f** option) and reply to the user "Hello username. You've checked a regular file"

Loop Constructs





for

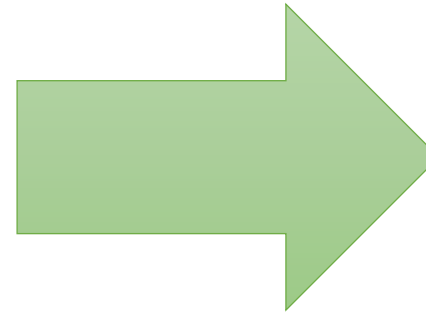


for

```
for <variable> in <list>  
  do <commands>  
done
```

Recap: Use Resources Freely Available to you

Share resources via Slack, for *(one of many)* example:



File Permissions

- Bash Ownership
- Bash Modify (chmod)
- Bash Ownership (chown)
- Bash Group (chgrp)

Scripting

Bash Syntax

- Bash Script
- Bash Variables
- Bash Data Types
- Bash Operators
- Bash If...Else
- Bash Loops
- Bash Functions
- Bash Arrays
- Bash Schedule (cron)

for

#!/bin/bash

Basic range in loop

for value in {1..5}

do

echo \$value

done

echo All done!

General format:

```
for var in varlist
do
    commands
done
```

The **for** statement executes the commands once for each value in the list

for example 1

```
# prints the numbers from 101 to 105
#
for num in 1 2 3 4 5
do
    expr $num + 100
done
```

Try out this weeks Lab

03: if, while, for, break & continue; case & select for menus; using sed editor

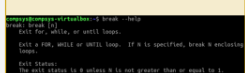
01: Conditionals and Loops



LOOPS REPEAT ACTIONS... SO YOU DON'T HAVE TO

test · if · for · while · until

02: Loop manipulation



break · continue · case · select

03: sed



Stream editor · find and replace

Lab



Loops · Menus · sed

02. Loops

for · while

for

Amend the basic **for** loop from the class notes to include a **for**, including an **if** statement and **break**

1. Look through this **for** code below: what is the expected output?

```
for i in {1..5};
do
  for j in {1..3};
  do
    if [[ $j -eq 2 ]]; then
      break
    fi
```


while

```
NUM=0  
while [ $NUM -lt 5 ]  
do  
echo "The number is "$NUM"."  
let NUM=$NUM+1  
done
```

Syntax

- general for using the Bash while loop is

while [condition]

do

//code to execute here

//increment/decrement

done

while

Programming languages have loop constructs to allow a block of code to be executed repeatedly, either a fixed number of times, or while some condition holds.

while instruction general format

```
while [ condition ]  
do  
    commands  
done
```

Example: The following script prints all numbers from 1 to 100

```
# Prints all numbers from 1 to 100  
#  
num=1  
while [ $num -le 100 ]  
do  
    echo $num  
    num=`expr $num + 1`  
done
```

file=*filename*

while read -r line;
do

echo \$line

done < "\$file"

Infinite loops

CTRL+C

- the **`true`** indicates the while loop will **forever** iterate

If it returns quickly, it's beneficial to insert a small pause between executions to avoid CPU load, eg

```
while true
do
    echo "I will continue to iterate unless you press Ctrl+C"
    echo "let me also rest 1 second between every iteration"
    sleep 1
done
```

`$#` Shell Variable

`echo $#` outputs the number of positional parameters of your script.

```
if [[ $# -ne 1 ]]; then
    echo "Please use only one
    argument for file name"
    exit 1
fi
```

while loop example:

```
#!/bin/bash
```

```
while [ "$#" -ne 1 ]; do
    #your commands here
done
```

until

```
# Until loop example  
count=1  
until [ $count -gt 5 ]; do  
    echo "Count is $count"  
    ((count++))  
done
```

until

until **CONDITION;**

do

CONSEQUENT-COMMANDS;

done

```
#!/bin/bash
```

```
counter=0
```

```
until [ $counter -ge 5 ]
```

```
do
```

```
    echo Counter: $counter
```

```
    ((counter++))
```

```
done
```

- Very similar to **while** loop
 - except that the loop executes until the **TEST-COMMAND** executes successfully

Try out this weeks Lab

03: if, while, for, break & continue; case & select for menus; using sed editor

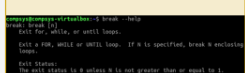
01: Conditionals and Loops



LOOPS REPEAT ACTIONS... SO YOU DON'T HAVE TO

test · if · for · while · until

02: Loop manipulation



break · continue · case · select

03: sed



Stream editor · find and replace

Lab



Loops · Menus · sed

Computer System... > 03: Loops, Men... > Week 3 Class 1... > Lab

EXERCISE

- Write the code, making use of a **for** loop to make a backup copy of each of your **.txt** files i.e. **filename.bak**

HINT consider listing and then redirecting all of your **.txt** files into some **temp** file; then consider copying each of these files as a new file extension)

while

Create a basic script to verify that the user has passed five arguments into it (Make use of class notes)

```
while [ "$#" -ne 5 ]; do
    #your commands here
    echo You must pass exactly five arguments please
    exit 1
done
```